算法设计与分析　课程材料

任务窃取调度

代码文档

Chase-Lev 双端队列 · 20 线程微基准 · 离散事件仿真器

接口说明 · 关键实现 · 构建运行 · 验证与移植

配套报告	《任务窃取调度》专题报告（课题 7）
代码范围	code/ws_deque.hpp、bench_ws.cpp、sim_ws.py、plot_results.py
验证方式	20 万项并发自测 · 三策略 checksum 交叉校验 · 第二种子复核
版本	v1.0 · 2026 年 11 月

这份文档说明课题 7 任务窃取专题的四个代码文件：它们各自解决什么问题、对外暴露什么接口、关键路径为什么这样写、怎样构建运行、如何验证正确性，以及搬进真实存储引擎时需要替换哪些部分。它不重复专题报告中的算法推导与实验结论，只回答“代码怎么用、为什么这样实现”。

代码遵循两条原则。第一，正确性优先：双端队列使用保守的 `seq_cst` 屏障，宁可损失一点性能，也不留下难以复现的内存序缺陷。第二，可验证：并发路径有 20 万项的恰好一次消费自测，三种策略处理同一批任务并比对 checksum；仿真结果与 C++ 微基准只做方向性对照，不互相证明绝对值。

目　录

1 概述

2 构建与运行

3 总体结构

4 ChaseLevDeque 实现

5 bench_ws.cpp 微基准

6 sim_ws.py 仿真器

7 plot_results.py 绘图

8 验证与测试

9 参数与调优

10 移植与扩展

附录 A ws_deque.hpp 完整清单

附录 B 命令速查

1 概述

1.1 目的与读者

本文档面向三类读者：需要评审实现的课程教师、准备把窃取线程池接进引擎的开发者、以及要复现实验数据的组员。对应的读法也不同——评审看第 4、8 节，开发者看第 5、9、10 节，复现看第 2、6、7 节。

四个文件构成两条独立的验证路径。C++ 路径在真实多核上回答“能不能跑对、快多少”；Python 路径在可控参数下回答“机制如何起作用”。两条路径共享同一套语义：所有者底部 LIFO、窃取者顶部 FIFO、阈值触发、失败即换目标。

1.2 文件清单

表 1 代码文件与规模

文件	语言	行数	类型	职责
ws_deque.hpp	C++17	123	头文件	Chase-Lev 双端队列，窃取的载体
bench_ws.cpp	C++17	362	可执行	20 线程三策略微基准 + 并发自测
sim_ws.py	Python 3	474	脚本	离散事件仿真：四策略、三负载、阈值与亲和实验
plot_results.py	Python 3	195	脚本	由结果 JSON 生成五张插图

1.3 运行环境

表 2 构建与运行环境

项目	要求	说明
C++ 编译器	g++ ≥ 8.1，C++17，-pthread	验证环境为 MinGW-w64 g++ 8.1.0；MSVC 2019 亦可
Python	≥ 3.10	仅用标准库 + numpy + matplotlib
第三方库	numpy ≥ 1.24，matplotlib ≥ 3.6	matplotlib 需能定位中文字体（Microsoft YaHei）
硬件	≥ 4 逻辑核，建议 20 核	C++ 基准默认取 hardware_concurrency，最多 64
操作系统	Windows 10/11 或 Linux	无平台特有 API；绑核与 NUMA 见第 10 节

2 构建与运行

2.1 编译 C++ 基准

```
# 在课题7_任务窃取/ 目录下
g++ -O2 -std=c++17 -pthread code/bench_ws.cpp -o bench_ws

# 若源文件与头文件不在同一目录，补上头文件搜索路径
g++ -O2 -std=c++17 -pthread -Icode code/bench_ws.cpp -o bench_ws
```

不需要链接 `libatomic`：队列元素是 `uint32_t`，原子操作可直接内联。若把元素换成 16 字节以上的结构体，GCC 可能生成 `__atomic_load` 等外部调用，需要追加 `-latomic`，这属于第 10.4 节列出的移植注意事项。

2.2 运行微基准

```
./bench_ws --selftest              # 并发正确性自测，约 1 秒
./bench_ws 2000000 20 skew         # 200 万任务、20 线程、Zipf 偏斜
./bench_ws 2000000 20 uniform      # 均匀访问
./bench_ws 2000000 20              # 第三个参数缺省时默认为 skew
```

位置参数依次是任务数、线程数、负载类型；也接受 `--skew` / `--uniform` 开关。线程数超过 64 或非正数时回退为 20，避免误传参数把机器压垮。

2.3 运行仿真与绘图

```
python code/sim_ws.py --out results/sim_main.json   # 主实验 + 阈值扫描 + 亲和实验
python code/plot_results.py                         # 生成 fig/fig1..fig5.png
```

完整仿真在本机约 3 分钟：三负载 × 三策略的主实验、7 档阈值扫描、2 负载 × 2 模式的亲和实验。输出除主文件外，还会在同目录生成 `sim_sweep.json` 与 `sim_affinity.json`；绘图脚本读取这三个文件，不重新仿真。

2.4 预期输出

```
SELFTEST OK: 200000 items, missing=0 duplicates=0
tasks=2000000 threads=20 workload=skew
policy      makespan_ms      Mop/s     P50_us     P99_us   P99.9_us     checksum
fifo              587.2      3.406   280788.0   581499.8   586488.5 1874899726873588722
local            1326.2      1.508   237349.3  1277092.6  1321440.0 1874899726873588722
ws                518.6      3.857   244907.3   511521.9   517880.2 1874899726873588722
checksum OK
```

上面是 2026-11-04 在本机（g++ 8.1.0，20 逻辑处理器）跑的一轮真实输出，原始日志见 `results/bench_rounds.log`；不同轮次会有 10%–20% 的波动，表 13 给的是 3 轮中位数。三行结果的 checksum 必须完全一致，否则说明某条路径漏算或重复计算了任务；程序会以非零码退出。所有分位数都不会超过 makespan：例如 ws 的 P99.9=517880.2 μs < 518.6 ms。

3 总体结构

3.1 两条验证路径

C++ 路径把任务预生成、屏障同步、计时与统计都握在手里，能得到真实缓存与 SMT 影响下的 makespan；缺点是参数（到达率、阈值、窃取成本）不能任意拨动，重复性也受操作系统调度干扰。Python 路径用离散事件模拟排队，能把“阈值从 2 调到 8”“窃取成本从 1 μs 调到 3 μs”单独隔离出来；代价是它不模拟锁、NUMA 与真实缓存，只能回答机制层面的问题。

因此报告中的定量结论都要求两条路径方向一致：仿真给出机制与趋势，C++ 基准给出真实多核上的量级与正确性。任何一条路径单独给出的数字都不应直接当作性能承诺。

3.2 模块依赖

表 3 模块依赖与输入输出

模块	依赖	输入	输出
bench_ws.cpp	ws_deque.hpp、pthread	任务数、线程数、负载类型	stdout 指标表 + 退出码
sim_ws.py	numpy	负载类型、λ、时长、阈值	JSON（config / rows / cdf）
plot_results.py	matplotlib、三个 JSON	results/*.json	fig/*.png
build_docx.py / build_code_doc.py	docx_kit.py、python-docx	results/*.json（人工核对）	两份 DOCX

3.3 术语约定

表 4 术语与代码映射

术语	含义	在代码中的体现
owner	分片的属主线程，唯一可以 push/pop 自己队列的线程	`deques[w]` 的 w
thief	队列为空、去别的队列顶部取任务的线程	`steal()` 的调用者
victim	被窃取的目标队列	随机探测得到的 `v`
shard	分片编号，属主为 `shard % P`	`OWNER[sh]`、`t.owner`
P	工作线程数	`P`、`threads`
k	窃取阈值，受害者长度 > k 才允许偷	`THRESH`、`thresh`
steals_per_k	每完成一千个任务的窃取次数	`steals / completed × 1000`
remote_pct	跨 NUMA 节点执行的任务比例	`m.remote / completed`

4 ChaseLevDeque 实现

4.1 设计约束与不变量

`ws_deque.hpp` 是一个只有 123 行的头文件，但它是整个窃取机制的地基。理解它只需要抓住四条不变量：

- **单所有者**：`push`、`pop` 只能由创建队列的线程调用；`steal` 可由任意其他线程调用。
- **两端分离**：常态下所有者只推进 `bottom`，窃取者只通过 CAS 推进 `top`；只有当队列只剩一个元素时，所有者和窃取者才会在 `top` 的 CAS 上相遇。
- **区间语义**：有效元素是 `[top, bottom)`；`pop` 先把 `bottom` 减一，使 `b` 指向最后一个元素，若 `t == b` 则与窃取者争用 CAS，失败时把 `bottom` 恢复。
- **旧数组不释放**：扩容后旧数组保留到队列析构，保证仍持有旧指针的窃取者不会访问已释放内存。

4.2 内部结构

表 5 ChaseLevDeque 的成员与访问权限

成员	类型	写入者	读取者	作用
`top_`	`atomic<size_t>`	所有者（CAS）、窃取者（CAS）	所有线程	队首下标，窃取侧边界

成员	类型	写入者	读取者	作用
bottom_	atomic<size_t>	仅所有者	所有线程	队尾下标，所有者侧边界
array_	atomic<Array*>	仅所有者（扩容时）	所有线程	当前环形数组，release 发布
arrays_	vector<unique_ptr<Array>>	仅所有者	仅所有者	持有全部历史数组，延迟回收
Array::cap	size_t	构造时	所有线程	容量，2 的幂
Array::buf	unique_ptr<atomic<T>[]>	元素写入	元素读取	环形缓冲区

4.3 公开接口

表 6 公开接口与线程约束

签名	调用者	语义	代价
ChaseLevDeque(size_t initial_capacity = 1024)	创建者（随后共享）	容量向上取整到 2 的幂	O(1)
void push(T v)	仅所有者	底部入队；满则扩容	摊还 O(1)，扩容 O(n)
bool pop(T& out)	仅所有者	底部出队；空或竞争失败返回 false	O(1)
bool steal(T& out)	任意其他线程	顶部出队；空或 CAS 失败返回 false	O(1)
size_t size() const	任意线程	近似长度，供阈值判断	O(1)

size() 是近似值：它先读 bottom 再读 top，两次读取之间队列可能变化。用于“是否超过阈值”这种启发式判断没有问题，但不能用于正确性决策。

4.4 三条路径

先看所有者入队。它先读 top 判断是否已满，把元素写入槽位，再用 release 屏障保证元素先于新的 bottom 可见，最后发布 bottom = b + 1。

```
void push(T value) {
    std::size_t b = bottom_.load(std::memory_order_relaxed);
    std::size_t t = top_.load(std::memory_order_acquire);
    Array* a = array_.load(std::memory_order_relaxed);
    if (b - t > a->cap - 1) {           // 满
        a = grow(a, b, t);
    }
    a->at(b).store(std::move(value), std::memory_order_relaxed);
    std::atomic_thread_fence(std::memory_order_release);
    bottom_.store(b + 1, std::memory_order_relaxed);
}
```

出队稍复杂：先减 `bottom` 预占槽位，插入 `seq_cst` 屏障后重读 `top`。如果 `t < b`，正常取出；如果 `t == b`，说明这是最后一个元素，必须用 CAS 与窃取者争夺。CAS 失败意味着被偷走，需要把 `bottom` 恢复回去。

```
bool pop(T& out) {
    std::size_t b = bottom_.load(std::memory_order_relaxed);
    std::size_t t = top_.load(std::memory_order_relaxed);
    if (b <= t) return false;           // 空（或正被窃取者取走最后一个）
    --b;
    Array* a = array_.load(std::memory_order_relaxed);
    bottom_.store(b, std::memory_order_relaxed);
    std::atomic_thread_fence(std::memory_order_seq_cst);
    t = top_.load(std::memory_order_relaxed);
    if (t <= b) {
        out = a->at(b).load(std::memory_order_relaxed);
        if (t == b) {                   // 只剩一个，与窃取者竞争
            if (!top_.compare_exchange_strong(t, t + 1,
                                              std::memory_order_seq_cst,
                                              std::memory_order_relaxed)) {
                bottom_.store(b + 1, std::memory_order_relaxed);
                return false;
            }
            bottom_.store(b + 1, std::memory_order_relaxed);
        }
        return true;
    }
    bottom_.store(b + 1, std::memory_order_relaxed);
    return false;
}
```

窃取路径最短：先 acquire 读 `top`，`seq_cst` 屏障后 acquire 读 `bottom`；若 `t < b`，读槽位并 CAS 推进 `top`。CAS 失败就返回 false——调用者应当换一个受害者，而不是原地重试同一个位置。

```
bool steal(T& out) {
    std::size_t t = top_.load(std::memory_order_acquire);
    std::atomic_thread_fence(std::memory_order_seq_cst);
    std::size_t b = bottom_.load(std::memory_order_acquire);
    if (t >= b) return false;
    Array* a = array_.load(std::memory_order_acquire);
    out = a->at(t).load(std::memory_order_relaxed);
    if (!top_.compare_exchange_strong(t, t + 1,
                                      std::memory_order_seq_cst,
                                      std::memory_order_relaxed)) {
        return false;
    }
    return true;
}
```

4.5 内存序为什么这样选

表 7 各路径的内存序与理由

操作	关键原子操作	内存序	原因
push	槽位写 → `bottom` 写	relaxed + release fence	保证元素先于新 `bottom` 可见
pop	`bottom` 预减 → 读 `top`	relaxed + seq_cst fence	与窃取者的 fence 形成全序，避免同时取走最后一个
steal	读 `top` → 读 `bottom` → CAS	acquire + seq_cst fence + acquire	先看队首再看队尾，配合屏障读到一致快照
size	读 `bottom`、`top`	relaxed	仅作启发式判断，允许误差
grow	复制元素 → 发布 `array_`	relaxed + release	新数组内容先于指针可见

这里采用的是 Lé 等人 2013 年给出的保守方案：所有者与窃取者在“最后一个元素”上使用 `seq_cst` 屏障，避免依赖更细的 acquire/release 组合。代价是每次 pop 多一条全屏障；在 20 线程、任务粒度 20 µs 的场景下，这条屏障的开销被服务时间完全掩盖。生产实现若要把窃取成本压到几十纳秒，可以按原论文的弱内存版本放宽，但必须重新做形式化验证与压力测试。

4.6 扩容与旧数组保留

扩容只在所有者路径发生：容量翻倍，把 `[t, b)` 的元素搬到新数组，再以 release 语义发布新指针。旧数组不释放，而是挂在 `arrays_` 里，直到队列析构。

```
Array* grow(Array* old, std::size_t b, std::size_t t) {
    auto na = std::make_unique<Array>(old->cap * 2);
    for (std::size_t i = t; i < b; ++i) {
        na->at(i).store(old->at(i).load(std::memory_order_relaxed),
                        std::memory_order_relaxed);
    }
    Array* raw = na.get();
    arrays_.push_back(std::move(na));   // 旧数组保留，避免悬垂访问
    array_.store(raw, std::memory_order_release);
    return raw;
}
```

为什么不能立即释放旧数组？窃取者可能已经读到了旧指针，正在读 `Array::cap` 或槽位；如果所有者在这时释放，就会发生 use-after-free。保留旧数组是最简单、也最容易证明正确的做法，代价是内存随扩容次数增长。若队列长期运行且容量反复翻倍，可以改用 epoch 或 hazard pointer 回收；本实现面向教学与基准测试，没有引入这层复杂度。

4.7 使用示例

```
ChaseLevDeque<uint32_t> q(1024);      // 所有者线程创建

// 所有者线程
q.push(task_id);
uint32_t v;
if (q.pop(v)) run(v);

// 任意其他线程（窃取者）
if (q.steal(v)) run(v);               // 失败就换下一个受害者
```

一个队列只能有一个所有者线程，但不同队列可以并行存在；窃取者数量不限。队列对象本身必须在所有线程停止访问后才能析构——通常由线程池的关闭流程保证。

4.8 复杂度与限制

表 8 操作复杂度与空间

操作	最好	摊还	最坏	说明
push	O(1)	O(1)	O(n)	触发扩容时复制活跃元素
pop	O(1)	O(1)	O(1)	最后一个元素多一次 CAS
steal	O(1)	O(1)	O(1)	失败即返回，不重试
size	O(1)	O(1)	O(1)	近似值

操作	最好	摊还	最坏	说明
空间	O(cap)	—	O(总扩容容量)	旧数组保留，最坏约为当前容量的 2 倍

- **元素类型**：`std::atomic<T>` 要求 `T` 可平凡复制；实践中建议存任务索引或指针，而不是大对象。
- **无公平性**：窃取者可能连续失败；算法只保证进展，不保证某个窃取者一定拿到任务。
- **无锁**：`push`、`pop`、`steal` 都不取锁；`push` 触发扩容时会分配内存，因此严格说它不是无等待的，但不会阻塞其他线程。
- **伪共享**：`top_` 与 `bottom_` 在对象中相邻，高争用下可能共享缓存行；本实现未加 `alignas(64)`，生产实现应把两者分开（见第 10 节）。

5 bench_ws.cpp 微基准

5.1 测量口径

基准先把 200 万个任务全部生成好，三种策略处理同一份 `TaskSet`；所有线程在 `StartGate` 上同步，`go` 置位后统一计时。makespan 取最后一个任务的完成时刻，吞吐为任务数除以 makespan。

这里的“延迟”是任务从统一 t0 到完成的批量周转时间，不是 open-loop 的排队延迟：没有到达过程，也就没有“等待被提交”的那段时间。因此 P99 只能用于三种策略之间的相对比较，不能当作 SLO 数字，更不能与仿真中的端到端延迟直接相加。

5.2 任务与负载生成

表 9 基准的任务参数

常量	取值	含义
`kShards`	80	分片数，属主为 `shard % threads`
`kTableBits`	13	每个分片 8192 项哈希表，制造真实内存访问
`kBaseProbes`	512	普通任务的相关哈希探测次数，约 4–7 µs
`kLongProbes`	16384	长任务探测次数，模拟 compaction 切片
`kLongFrac`	1%	长任务比例

`run_task` 用指针追逐的方式访问分片表：每次探测都依赖上一次的哈希值，无法被编译器向量化，接近真实 KV 引擎中“查索引—取 value—再哈希”的访存模式。任务的分片决定属主，属主决定它默认进入哪个队列。

```
for (std::size_t i = 0; i < n; ++i) {
    Task t{};
    t.key = keys(rng);
    t.id = static_cast<uint32_t>(i);
    t.shard = (workload == "skew") ? zipf(rng) : uni(rng);
    t.owner = t.shard % threads;
    t.is_long = (u01(rng) < kLongFrac) ? 1 : 0;
    ts.tasks[i] = t;
    ts.by_owner[t.owner].push_back(static_cast<uint32_t>(i));
}
```

5.3 三种策略

表 10 三种策略的实现差异

策略	任务来源	调度方式	迁移	预期
集中式 FIFO	全局原子游标，一次取 8 个	谁空谁取，天然均衡	每个任务都可能跨核	均衡好、局部性差、游标争用
本地队列	`by_owner[w]` 顺序处理	只看自己的分片	无	局部性好、失衡时木桶效应
工作窃取	自己的 deque，空了偷别人	底部 LIFO + 顶部 FIFO	按需、低频	兼顾均衡与局部性

集中式 FIFO 用 `fetch_add` 批量领号，批大小 8 是争用与公平的折中：批量太小，20 个核会挤在同一个游标上；批量太大，末尾会出现“一个线程领走最后一批、其他线程空转”的长尾。

```
constexpr std::size_t kBatch = 8;
for (;;) {
    std::size_t begin = cursor.fetch_add(kBatch, std::memory_order_relaxed);
    if (begin >= n) break;
    std::size_t end = std::min(begin + kBatch, n);
    uint64_t sum = 0;
    for (std::size_t i = begin; i < end; ++i) {
        sum += run_task(ts.tasks[i], tables);
        latency[i] = now_ns() - t0;
        done.fetch_add(1, std::memory_order_relaxed);
    }
    checksum.fetch_add(sum, std::memory_order_relaxed);
}
```

工作窃取先做一次预填充：每个属主把自己的任务压入自己的队列，这一步不计时，因为真实系统里任务也是先进入注入队列再由属主取走。计时开始后，所有者先 `pop`，失败则随机探测最多 `2P` 个受害者，成功即执行，失败则 `yield` 后重试；`remaining` 归零时所有线程退出。

```
while (remaining.load(std::memory_order_relaxed) > 0) {
    uint32_t id = 0;
    bool got = false;
    if (deques[w]->pop(id)) {
        got = true;
    } else {
        for (int probe = 0; probe < threads * 2 && !got; ++probe) {
            int v = static_cast<int>(rng() % threads);
            if (v != w && deques[v]->steal(id)) {
                got = true;
                steals.fetch_add(1, std::memory_order_relaxed);
            }
        }
    }
    if (!got) {
        if (remaining.load(std::memory_order_relaxed) == 0) break;
        std::this_thread::yield();
        continue;
    }
    const Task& t = ts.tasks[id];
    sum += run_task(t, tables);
    latency[t.id] = now_ns() - t0;
    remaining.fetch_sub(1, std::memory_order_relaxed);
}
```

`remaining` 同时承担“还有多少任务”和“何时退出”两个职责：每完成一个任务减一，归零后仍在探测的线程在下一次循环直接跳出。这比维护活跃线程计数简单，也不会出现“最后一个任务被偷走、原线程已退出”的竞态。

5.4 指标与校验

表 11 基准输出字段

字段	含义	用途
`makespan_ms`	首个任务开始到最后一个任务结束	策略对比的主指标
`throughput_mops`	完成任务数 / makespan	换算成每秒百万任务
`p50/p99/p999_us`	批量周转时间的分位数	观察尾部长尾，不作 SLO
`checksum`	所有任务返回值的和	三策略一致性校验
`processed`	完成任务数	应等于任务总数
`steals`	成功窃取次数	衡量迁移频率

checksum 是最后一道防线：三种策略如果漏算、重算或把同一个任务执行两次，校验和几乎必然不同。程序在打印结果后比对三者，不一致就返回 1。

5.5 命令行参数

表 12 bench_ws 命令行参数

参数	默认	说明
[任务数]	2000000	第一个位置参数
[线程数]	hardware_concurrency()	超过 64 或非正数时回退 20
[uniform|skew]	skew	第三个位置参数，也可用 --uniform / --skew
--selftest	—	运行并发自测后退出，忽略其他参数

5.6 实测结果

表 13 20 线程微基准结果（200 万任务，3 轮中位数；相对窃取=本策略/窃取）

负载	策略	makespan（ms）	吞吐（Mtask/s）	相对窃取
均匀	集中式 FIFO	772.0	2.590	1.49×
均匀	本地队列	495.0	4.040	0.96×
均匀	工作窃取	516.5	3.872	1.00×
偏斜	集中式 FIFO	571.7	3.498	1.12×
偏斜	本地队列	1326.2	1.508	2.59×
偏斜	工作窃取	511.4	3.910	1.00×

均匀负载下窃取的 makespan 比 FIFO 短 33%，但比本地队列慢 4%（516.5 ms 对 495.0 ms）。FIFO 慢的原因是全局游标的原子操作与缓存行争用，以及任务被随机分给任意核后分片表不在本地缓存；本地队列与窃取在均匀负载下都没有明显失衡，差异主要来自窃取的探测/迁移开销。

偏斜负载下差距拉大：本地队列的 makespan 是窃取的 2.59 倍（1326.2 ms 对 511.4 ms）。FIFO 仍比窃取慢 12%，同样是因为它把任务均衡地分给了 20 个核，却让每个核都去访问不在本地的分片表。两轮之间绝对值会变（例如 FIFO 均匀 763–772 ms），因此表 13 只给 3 轮中位数，比较时看同一轮内的相对关系。

5.7 偏差与限制

- **批量延迟**：没有到达过程，分位数反映的是“从统一起点到完成”，不是请求级 SLO。

- **未绑核**：线程由 OS 调度，可能迁移；SMT 邻居与后台进程会带来波动。本机同配置 3 轮中，FIFO 差约 1%，本地队列（均匀）差约 21%，窃取差约 10%–15%，所以只报中位数。
- **预填充不计时**：提交与入队成本被排除在 makespan 之外，真实系统要把注入队列算进去。
- **长任务主导尾部**：1% 的长任务是 P99.9 的主要来源，改变长任务比例会显著改变尾部分位数。

6 sim_ws.py 仿真器

6.1 模型参数

表 14 仿真模型参数

参数	取值	说明
P	20	工作线程数，分 2 个 NUMA 节点，每节点 10 核
SHARDS	80	20 × 4，分片 s 的属主为 s mod 20
CAP	32768 / 队列	有界队列，满则丢弃
MEAN_SVC	20 µs	基础服务时间均值
SIGMA	0.5	对数正态形状参数
LONG_FRAC / LONG_MEAN	1% / 300 µs	长任务比例；在基础服务时间上额外叠加 U(180, 420) µs
STEAL_COST	1 µs	一次窃取的 CAS + 冷启动开销
FIFO_OVERHEAD	0.3 µs	集中式队列每次操作的争用开销
THRESH	2	受害者队列长度 > k 才允许窃取
PROBES	8	每次空闲最多探测的受害者数量（实现中取 2×）
HORIZON	500000 µs	仿真时长，0.5 秒
UTIL	0.8	目标利用率，λ = 0.8 × 容量

容量按 `P / (MEAN_SVC + LONG_FRAC × LONG_MEAN)` 估算，约 0.87 Mtask/s；到达率取它的 80%，即 0.696 Mtask/s。所有时间单位都是微秒，绘图时再换算成毫秒。

6.2 代码结构

表 15 仿真器的主要函数

函数 / 类	职责
`zipf_weights(n, theta)`	生成 Zipf(θ) 权重，默认 θ = 1.1
`sample_services(rng, n)`	对数正态服务时间，叠加 1% 的长任务

函数 / 类	职责
`build_workload(kind, lam, horizon, rng)`	生成 `(times, shards, services)`，按时间升序
`Metrics`	累计延迟、完成数、丢弃数、窃取数、远程数，输出 summary 与 CDF
`make_idle_helpers()`	空闲线程集合与 O(1) 取用/归还
`simulate_fifo`	全局队列 + 20 个服务台的离散事件
`simulate_local`	每属主一个队列，无迁移
`simulate_ws`	工作窃取，可选阈值、远程惩罚与亲和
`run_main` / `run_sweep` / `run_affinity`	三组实验，写 JSON

6.3 负载生成

均匀负载用泊松到达、均匀分片；偏斜负载用 Zipf(1.1) 抽样分片，最热分片约占 24% 的请求；动态热点把时间切成 25 ms 一段，每段把 `hot_cycle` 中的一个分片权重乘 3，模拟“热点轮换”。

```
weights = zipf_weights(SHARDS)
hot_cycle = [0, 7, 3, 19, 11, 42, 5, 28]
step = 25000.0
t, seg = 0.0, 0
while t < horizon:
    end = min(t + step, horizon)
    n = rng.poisson(lam * (end - t))
    if n > 0:
        ts = np.sort(rng.uniform(t, end, n))
        w = weights.copy()
        w[hot_cycle[seg % len(hot_cycle)]] *= 3.0
        w /= w.sum()
        sh = np.searchsorted(np.cumsum(w), rng.random(n))
        times.append(ts)
        shards.append(np.minimum(sh, SHARDS - 1))
    t = end
    seg += 1
```

服务时间先按对数正态抽样，再随机挑 1% 的任务加上 180–420 μs 的额外工作。长任务不是“异常”，而是 compaction、flush、批量回填这类真实后台任务的抽象；它们决定了尾延迟的下限。

6.4 四种策略

表 16 四种仿真策略

策略	队列结构	窃取触发	远程惩罚	用途
`fifo`	一个全局 deque + 20 个服务台	无（空闲即取队首）	无	集中式基线
`local`	每属主一个 deque	无	无	纯分片基线
`ws`	每属主一个 deque	阈值 + 空闲探测	可开关	工作窃取
`ws_affinity`	同上	同节点优先，跨节点兜底	35%	亲和感知窃取

`simulate_ws` 有两条窃取路径。第一条是拉取：某线程完成一个任务、自己的队列为空时，随机探测受害者，若其长度超过阈值就从顶部取一个。第二条是帮助式移交：新任务入队后若属主队列已超过阈值且存在空闲线程，直接把队首交给空闲线程，减少“等属主完成再唤醒”的延迟。两条路径都计入窃取次数，并各自付出 `STEAL_COST`。

6.5 事件循环

```
for t in arrivals:                      # 时间升序
    while heap and heap[0].time <= t:   # 先处理到期完成事件
        complete(heap.pop())
        if 本线程队列非空: 继续执行
        else: 探测受害者并窃取，或标记空闲

    sh = shard[t]; ow = OWNER[sh]
    if ow 空闲: 立即执行
    elif 队列未满: 入队
    else: dropped += 1
    if 有空闲线程 and len(ow 队列) > k: 移交队首

while heap 非空: 处理完成事件，重复上述窃取逻辑
断言: 队列残留为 0 且 completed + dropped == arrivals；否则报错
```

事件堆里存的是 **(完成时刻，线程)**；同一时刻有多个完成事件时，按堆序依次处理。这个顺序不影响统计量的期望，但会影响具体某条延迟，因此仿真固定随机种子以保证可复现。

6.6 输出 JSON

表 17 结果 JSON 的结构

层级	字段	含义
`config`	`P`、`shards`、`capacity_cap`、`lambda`、`horizon`、`thresh`、`seed`	本次运行的参数

层级	字段	含义
rows[]	arrivals、accepted、completed、window_completed、dropped、backlog_at_horizon	计数与窗口积压
rows[]	makespan_us、throughput_mops（窗口）、drain_mops（排空）、mean_us、p50/p90/p99/p999_us	性能指标
rows[]	drop_rate、window_completion_pct、steals、steals_per_k、remote_pct、bins	丢弃、迁移与远程执行
cdf	"uniform|fifo" → [[q, latency], …]	绘制 CDF 的 300 个分位点

bins 是每 10 ms 的完成计数，用于画动态热点的吞吐时序；cdf 只在主实验里生成，避免文件过大。阈值扫描与亲和实验的输出是 rows 数组，结构与主实验的 rows 相同。

6.7 运行与结果

主实验的三组数字已在专题报告中列出：均匀负载三种策略的窗口完成率都在 0.696 Mtask/s 左右，P99 分别为 154.7 / 880.0 / 229.1 μs；偏斜负载本地队列跌到 0.497 Mtask/s、丢弃 10%、到 0.5 s 时积压约 6.4 万，P99 759 ms，窃取恢复到 0.696、251.7 μs；动态热点下窃取 0.697、252.3 μs，本地队列 0.483、12.8% 丢弃。throughput_mops 是固定窗口内完成数，drain_mops 才是“完成数 / makespan”；旧文档把后者当吞吐，会把本地队列算成 0.25。

7 plot_results.py 绘图

绘图脚本不重新仿真，只读取三个 JSON 并输出 5 张 PNG（200 dpi）。它把中文字体设为 Microsoft YaHei，负号设为 ASCII，避免图中出现方框或乱码。

表 18 五张插图的数据来源

图	数据来源	内容
fig1_overview.png	sim_main.json rows	三负载 × 三策略的吞吐与 P99 柱状对比
fig2_cdf_skew.png	sim_main.json cdf	偏斜负载的延迟 CDF（0.5–0.9999）
fig3_dynamic.png	sim_main.json bins	动态热点的吞吐时序，每 10 ms 一个点
fig4_threshold.png	sim_sweep.json	阈值 1–64 对 P99 与窃取率的影响
fig5_affinity.png	sim_affinity.json	随机窃取与亲和窃取的远程比例、P99

所有图都使用同一套配色：FIFO 蓝、本地队列橙、工作窃取绿；坐标轴标签与图例均为中文。若更换数据集，只需保持 JSON 的字段名不变，脚本无需修改。

8 验证与测试

8.1 并发自测

`bench_ws --selftest` 启动 1 个所有者与 4 个窃取者，共享一个初始容量为 4 的队列。所有者压入 20 万项，每压 1000 项顺手弹出一个；4 个窃取者不停 `steal`，直到主线程把剩余项全部弹出、`stop` 置位。每个被消费的项都写入一个原子计数数组，最后逐项检查：`missing == 0`、`duplicates == 0`、`consumed == 200000`。只统计总数无法发现“丢 1 个、重 1 个”的故障，逐项计数才能。

这个测试专门覆盖三个最容易出错的点：队列为空时的 `steal`、最后一个元素的 CAS 竞争、以及扩容期间窃取者读取旧数组。任何一处写错，都会表现为“少消费”或“重复消费”，测试立刻失败。

8.2 校验和交叉验证

微基准让三种策略处理同一批任务，并累加每个任务的返回值。若某条路径漏算、重算或把任务分错，校验和几乎必然不同。程序在打印三行结果后比对，不一致即返回非零码；正常输出末尾的 `checksum OK` 就是通过标志。

8.3 重复性与第二种子的复核

仿真用固定种子 20261104 生成主结果，再用种子 7 完整重跑。P99 最大偏差 6.3%（dynamic/FIFO 186.4→174.6 μs），P50 最大偏差 10.8%（dynamic/本地 4.3→4.8 ms），窗口完成率偏差在 1% 以内。所以文档不再宣称“偏差均小于 5%”；C++ 基准同配置 3 轮的 makespan 差可达 21%，因此只报中位数、并附原始日志，比较时只看同一轮内的相对值。

8.4 验证矩阵

表 19 验证矩阵

测试项	命令	判据	结果
队列恰好一次	`./bench_ws --selftest`	missing=0 且 duplicates=0	通过
三策略一致	`./bench_ws 2000000 20 skew`	三个 checksum 相同	通过

测试项	命令	判据	结果
均匀负载	`./bench_ws 2000000 20 uniform`	吞吐与 makespan 在合理区间	通过
仿真可复现	`python code/sim_ws.py --out …`	同种子结果一致	通过
第二种子	`sim_ws.run_main(..., seed=7)`	方向一致，记录偏差	通过
C++ 可追溯	`bench_rounds.log`	3 轮原始日志与中位数	通过
图与数据一致	`python code/plot_results.py`	5 张 PNG 均生成	通过

9 参数与调优

表 20 关键参数与调优建议

参数	位置	默认	影响	建议
`THRESH` / `thresh`	sim	2	越大迁移越少、排队越长（bench 无阈值，队列空即偷）	延迟优先 1–2；折中 4 到 8；SLO 1 ms 时不超过 8
探测上限	bench：2P；sim：2×PROBES	40 / 16	越大越容易找到活，但空转更久	保持 O(P)，全局空闲时尽快退避
`CAP`	sim	32768	越小越早丢任务，越大则尾延迟越高	按 SLO 与内存预算定，不建议无界
`STEAL_COST`	sim	1 µs	直接抬高每次迁移成本	按实测 CAS + 冷启动校准，跨节点再乘 1.4 到 2 倍
`FIFO_OVERHEAD`	sim	0.3 µs	集中式队列的争用开销	核数越多取值越大，64 核可到 1 µs 以上
`LONG_FRAC`	sim	1%	决定尾延迟下限	按真实 compaction 频率设置
`kBaseProbes`	bench	512	单任务服务时间	约 4–7 µs，接近批量化点查

调参的顺序应当是“先定粒度、再定阈值、最后调探测”。粒度决定窃取是否有意义：单任务远小于窃取成本时，应先把任务聚合；阈值决定排队与迁移的平衡；探测上限只影响全局空闲时的空转开销，不应该用来弥补粒度问题。

10 移植与扩展

10.1 接入真实线程池

真实引擎不会让业务线程直接操作双端队列，而是包一层工作窃取线程池。提交路径通常是：生产者把任务放进注入队列，工作线程一次取 32–64 个压入自己的 deque。批量注入把全局争用降一个数量级；如果注入队列用锁，批量是必须的，否则它会成为新的瓶颈。

线程池还需要决定四件事：线程是否绑核（应当绑，否则窃取省下的局部性会被 OS 迁移吃掉）；是否允许嵌套提交（允许时要帮助执行被 join 的子任务，避免死锁）；关闭时如何排空（先停止接收，再让窃取把剩余任务做完）；以及如何暴露指标（见 10.3）。

10.2 NUMA 与绑核

本实现不包含平台相关代码。移植到 Linux 时，常用组合是：`pthread_setaffinity_np` 绑核、`numa_alloc_onnode` 分配分片内存、`numa_run_on_node` 限制执行节点。窃取策略上，把受害者选择改成“同 LLC → 同节点 → 跨节点”三级，跨节点设置更长的冷却期与更低的预算；这与仿真中的亲和策略一一对应。

绑定线程后要重新测量窃取成本：跨节点一次远程访问可能是本地的 1.4–2 倍，原来 1 µs 的窃取可能变成 2 µs，阈值也应随之调整。

10.3 监控指标

- **steals/s 与 steal_fail/s**：区分“没活可偷”和“偷不到”，前者说明全局空闲，后者说明争用激烈。
- **每核队列长度**：定位热点与阈值是否合适；长期接近容量说明需要背压或扩核。
- **remote_pct**：跨节点执行比例，配合 `numastat` 判断亲和是否被破坏。
- **P99/P99.9 与长任务占比**：尾延迟恶化时，先看是不是长任务变多，而不是先调阈值。
- **迁移后二次迁移率**：短时间内被再次偷走的比例，衡量抖动。

10.4 已知限制

表 21 已知限制与处理方式

限制	影响	处理方式
旧数组不回收	内存随扩容次数增长	epoch / hazard pointer；或预估容量一次性分配
`top_`/`bottom_` 未分开缓存行	高争用下伪共享	`alignas(64)` 或填充字段

限制	影响	处理方式
元素要求可平凡复制	不能直接存大对象	存索引或指针，对象放任务池
无批量窃取	任务极短时窃取开销占比高	一次偷一半（见报告 7.3 节）
无优先级	后台任务可能挤占前台	任务打可偷标记，受害者前台队列过长时禁止偷
无公平性保证	某个窃取者可能长时间失败	随机化 + 退避；必要时限制单线程探测次数
亲和只作用于派发	探测路径仍跨节点，远程比例降不到零	探测也按同节点优先；跨节点冷却/预算
动态热点证据有限	静态/动态 P99 接近，不能单独证明“跟踪”	按 bin 统计每核队列与热点迁移，增加轮换频率
单所有者	不能多线程共同 push	每线程一个队列；跨线程提交走注入队列

附录 A　ws_deque.hpp 完整清单

以下为 code/ws_deque.hpp 的完整内容（123 行），代码与仓库中的文件一致；行首缩进在文档中以段落缩进呈现，复制代码时请以 code/ 下的源文件为准。阅读时重点关注三处：pop 中 t == b 的 CAS、grow 中对旧数组的保留、以及三个路径各自的屏障位置。

```
// ws_deque.hpp —— Chase-Lev 工作窃取双端队列（C++17 参考实现）
//
// 语义：
//   * 所有者线程调用 push() / pop()，只在“底部”操作，LIFO；
//   * 任意窃取者线程调用 steal()，只在“顶部”操作，FIFO，CAS 竞争；
//   * 所有者是唯一写 bottom 的线程，窃取者只写 top。
//
// 说明：本实现采用 Lé 等人 (2013) 的简化内存序（seq_cst fence），
// 以正确性优先、便于教学；生产实现可进一步放宽屏障。
// 队列按 2 的幂扩容，旧数组保留到析构，避免窃取者访问已释放内存。

#pragma once

#include <atomic>
#include <cstddef>
#include <memory>
#include <utility>
#include <vector>

template <class T>
class ChaseLevDeque {
public:
    explicit ChaseLevDeque(std::size_t initial_capacity = 1024) {
        std::size_t cap = 2;
        while (cap < initial_capacity) cap <<= 1;
        arrays_.push_back(std::make_unique<Array>(cap));
        array_.store(arrays_.back().get(), std::memory_order_relaxed);
    }

    ChaseLevDeque(const ChaseLevDeque&) = delete;
    ChaseLevDeque& operator=(const ChaseLevDeque&) = delete;
    ~ChaseLevDeque() = default;

    // 仅所有者线程调用
    void push(T value) {
        std::size_t b = bottom_.load(std::memory_order_relaxed);
        std::size_t t = top_.load(std::memory_order_acquire);
        Array* a = array_.load(std::memory_order_relaxed);
        if (b - t > a->cap - 1) {           // 满
            a = grow(a, b, t);
        }
        a->at(b).store(std::move(value), std::memory_order_relaxed);
        std::atomic_thread_fence(std::memory_order_release);
        bottom_.store(b + 1, std::memory_order_relaxed);
    }

    // 仅所有者线程调用；返回 false 表示队列为空
    bool pop(T& out) {
        std::size_t b = bottom_.load(std::memory_order_relaxed);
```

```
        std::size_t t = top_.load(std::memory_order_relaxed);
        if (b <= t) return false;           // 空（或正被窃取者取走最后一个）
        --b;
        Array* a = array_.load(std::memory_order_relaxed);
        bottom_.store(b, std::memory_order_relaxed);
        std::atomic_thread_fence(std::memory_order_seq_cst);
        t = top_.load(std::memory_order_relaxed);
        if (t <= b) {
            out = a->at(b).load(std::memory_order_relaxed);
            if (t == b) {                   // 只剩一个，与窃取者竞争
                if (!top_.compare_exchange_strong(t, t + 1,
                                                  std::memory_order_seq_cst,
                                                  std::memory_order_relaxed)) {
                    bottom_.store(b + 1, std::memory_order_relaxed);
                    return false;
                }
                bottom_.store(b + 1, std::memory_order_relaxed);
            }
            return true;
        }
        bottom_.store(b + 1, std::memory_order_relaxed);
        return false;
    }

    // 任意线程调用；返回 false 表示队列为空或竞争失败
    bool steal(T& out) {
        std::size_t t = top_.load(std::memory_order_acquire);
        std::atomic_thread_fence(std::memory_order_seq_cst);
        std::size_t b = bottom_.load(std::memory_order_acquire);
        if (t >= b) return false;
        Array* a = array_.load(std::memory_order_acquire);
        out = a->at(t).load(std::memory_order_relaxed);
        if (!top_.compare_exchange_strong(t, t + 1,
                                          std::memory_order_seq_cst,
                                          std::memory_order_relaxed)) {
            return false;
        }
        return true;
    }

    // 近似长度，仅用于阈值判断与统计
    std::size_t size() const {
        std::size_t b = bottom_.load(std::memory_order_relaxed);
        std::size_t t = top_.load(std::memory_order_relaxed);
        return b > t ? b - t : 0;
    }

private:
    struct Array {
        std::size_t cap;
        std::unique_ptr<std::atomic<T>[]> buf;
        explicit Array(std::size_t c)
            : cap(c), buf(std::make_unique<std::atomic<T>[]>(c)) {}
        std::atomic<T>& at(std::size_t i) { return buf[i & (cap - 1)]; }
        const std::atomic<T>& at(std::size_t i) const { return buf[i & (cap - 1)]; }
    };

    Array* grow(Array* old, std::size_t b, std::size_t t) {
```

```
        auto na = std::make_unique<Array>(old->cap * 2);
        for (std::size_t i = t; i < b; ++i) {
            na->at(i).store(old->at(i).load(std::memory_order_relaxed),
                            std::memory_order_relaxed);
        }
        Array* raw = na.get();
        arrays_.push_back(std::move(na));   // 旧数组保留，避免悬垂访问
        array_.store(raw, std::memory_order_release);
        return raw;
    }

    std::atomic<std::size_t> top_{0};
    std::atomic<std::size_t> bottom_{0};
    std::atomic<Array*> array_{nullptr};
    std::vector<std::unique_ptr<Array>> arrays_;   // 仅所有者线程修改
};
```

附录 B　命令速查

表 22　常用命令

任务	命令
编译基准	`g++ -O2 -std=c++17 -pthread code/bench_ws.cpp -o bench_ws`
并发自测	`./bench_ws --selftest`
偏斜负载	`./bench_ws 2000000 20 skew`
均匀负载	`./bench_ws 2000000 20 uniform`
主仿真	`python code/sim_ws.py --out results/sim_main.json`
快速冒烟	`python code/sim_ws.py --quick`
生成插图	`python code/plot_results.py`
生成报告	`python build_docx.py`
生成代码文档	`python build_code_doc.py`